WEEK 1 TASK

# Project Planning & System Architecture

StudySync: a collaborative study-group platform

| Item | Details |
|---|---|
| Prepared by | Divyanshi Katiyar |
| Role | Junior Full Stack Developer Intern |
| Deliverable | Project brief, user flowcharts, wireframes, technical architecture, API design, database schema, design rationale |
| Proposed stack | MongoDB, Express.js, React, Node.js (MERN) with Tailwind CSS and Socket.io |
| Date | 20 October 2026 |

## Contents

1. Project Brief
2. Technology Stack and Rationale
3. User Flowcharts
4. Wireframes
5. Technical Architecture
6. REST API Design
7. Database Design
8. Design Decisions and Rationale
9. Risks, Roadmap and Conclusion

# 1. Project Brief

## 1.1 Overview and purpose

**StudySync** is a full stack web application that lets students form study groups around a subject or exam, share learning resources, hold structured discussions and schedule study sessions in one place. Its purpose is to replace the scattered mix of chat groups, shared drives and calendar invites that students use today with a single focused workspace.

## 1.2 Problem statement

Students who study together currently juggle several tools. Resources get buried in chat history, nobody is sure when the next session is, new members cannot find past material, and there is no easy way to discover groups outside one's own circle. The result is duplicated effort, missed sessions and lost notes.

## 1.3 Target users

| Persona | Description | Main needs |
|---|---|---|
| Student (Alex) | Final-year student preparing for exams and placements | Find or create a group, access shared notes quickly, never miss a session |
| Group leader (Priya) | Student who organises a group and moderates content | Invite members, manage roles, schedule sessions, keep resources tidy |
| Platform admin | Staff member or senior student who oversees the platform | Review reported content, remove abusive groups, view platform statistics |

## 1.4 Project objectives

- Deliver a working MVP where a user can register, create or join a group, share resources and schedule sessions within one visit.
- Provide role-based access so private group content is visible only to its members.
- Keep core pages responsive on screens from 320 px to desktop widths.
- Use a layered, well-documented REST API so the front end and back end can be developed and tested independently.
- Apply security basics from day one: hashed passwords, token-based auth, input validation and rate limiting.

## 1.5 Functional requirements

Requirements are prioritised with the **MoSCoW** method (Must, Should, Could, Won't) so that the MVP scope stays realistic.

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Register with name, email and password; validate input; reject duplicate emails | Must |
| FR-02 | Log in and out using JWT access and refresh tokens | Must |
| FR-03 | View and edit own profile (name, avatar) | Should |
| FR-04 | Create a group with name, description, subject and public/private visibility | Must |
| FR-05 | Discover public groups with search, subject filter, sorting and pagination | Must |

| ID | Requirement | Priority |
|---|---|---|
| FR-06 | Join a public group, or any group using an invite code; leave a group | Must |
| FR-07 | Role-based permissions inside a group (owner, moderator, member) | Must |
| FR-08 | Share resources (link, file or note) with tags; list, filter and delete them | Must |
| FR-09 | Group discussion: create posts and comment on them | Should |
| FR-10 | Schedule study sessions with date, time and meeting link; members RSVP | Must |
| FR-11 | In-app notifications delivered in real time (new resource, session reminder, new member) | Should |
| FR-12 | Personal dashboard summarising groups, upcoming sessions and recent activity | Must |
| FR-13 | Email reminders before a session | Could |
| FR-14 | Admin panel to moderate groups and content | Could |
| FR-15 | Native mobile app and video calling inside the platform | Won't (this release) |

## 1.6 Non-functional requirements

| Area | Requirement |
|---|---|
| Performance | List endpoints are paginated; target response time under 300 ms for typical requests; pages load in under 3 s on a normal broadband connection |
| Security | bcrypt password hashing, short-lived JWT access tokens, httpOnly refresh cookie, HTTPS, CORS whitelist, rate limiting, schema validation and NoSQL-injection sanitising |
| Usability | Mobile-first responsive layout, consistent navigation, clear error and empty states, loading skeletons |
| Accessibility | Semantic HTML, keyboard navigation, visible focus, colour contrast meeting WCAG 2.1 AA for text |
| Scalability | Stateless API so several instances can run behind a load balancer; indexes on every frequent query |
| Maintainability | Layered folder structure, ESLint and Prettier, environment-based config, unit and API tests for core flows |
| Reliability | Centralised error handling, consistent JSON error format, automated Atlas backups |
| Compatibility | Latest two versions of Chrome, Firefox, Edge and Safari |

## 1.7 Scope

**In scope:** everything marked Must or Should in section 1.5, deployed as a single web application.

**Out of scope:** native apps, built-in video calls, payments, AI-generated study material, and integration with university systems. These are listed as possible future work.

## 1.8 Key user stories

| As a... | I want to... | So that... |
|---|---|---|
| new visitor | create an account with my email | I can join study groups |
| student | search public groups by subject | I can find people studying the same topic |
| group member | upload and tag a resource | others can find it later without scrolling chat history |
| group leader | schedule a session and see who has RSVP'd | I know how many people will attend |
| student | get a notification when something new is posted | I do not have to keep checking each group |
| group leader | promote a member to moderator | I can share the work of keeping the group tidy |

## 1.9 Success measures

- A new user can reach a working group page in under 2 minutes from the landing page.
- All Must requirements pass their acceptance tests.
- No unauthorised user can read or write a private group's data (verified by API tests).
- Lighthouse scores of 85 or above for performance and accessibility on the main pages.

## 2. Technology Stack and Rationale

| Layer | Choice | Why this choice | Alternative considered |
|---|---|---|---|
| Front end | React 18 with Vite, React Router, Tailwind CSS | Component model suits repeated UI (cards, lists); fast dev server; Tailwind speeds up consistent, responsive styling | Vue, Angular (steeper setup for a small team) |
| Server state | React Query and Axios | Handles caching, loading and error states for API data with little code; interceptors manage token refresh | Redux Toolkit (more boilerplate for mostly server data) |
| Back end | Node.js with Express | Same language (JavaScript) across the stack; large ecosystem; simple REST routing and middleware | NestJS (heavier), Django (different language) |
| Database | MongoDB with Mongoose | Flexible documents fit resources and posts; Mongoose adds schemas, validation and population; Atlas gives a free managed tier | PostgreSQL (strong for relations, but more setup for this MVP) |
| Real time | Socket.io | Rooms map naturally to groups; automatic reconnection and fallback | Server-Sent Events (one-way only) |
| Auth | JWT access token + refresh token in httpOnly cookie | Stateless API; refresh cookie reduces XSS exposure | Server sessions (need shared store when scaling) |
| Validation | Zod (server) and React Hook Form (client) | One schema language, clear error messages | Joi, Yup |
| Files | Cloudinary | Offloads storage and delivery; signed uploads; free tier | Local disk (not durable on cloud hosts), AWS S3 (more setup) |
| Hosting | Vercel (client), Render (API), MongoDB Atlas (DB) | Free tiers, Git-based deploys, HTTPS by default | Single VPS (more manual work) |
| Testing and tools | Jest, Supertest, Postman, ESLint, Prettier, Git and GitHub | Covers unit and API testing and keeps code style consistent | - |

# 3. User Flowcharts

Two flowcharts describe the most important journeys. Rounded green shapes are start and end points, blue rectangles are actions, amber diamonds are decisions, red boxes are error states, and green rectangles show the API call behind an action.

## 3.1 Registration and login

A visitor either registers or logs in. Registration validates input and checks that the email is unique before the password is hashed with bcrypt. After a successful login the server issues an access token and a refresh token, and the user lands on the dashboard. Wrong credentials return a generic message so the system does not reveal which emails are registered.

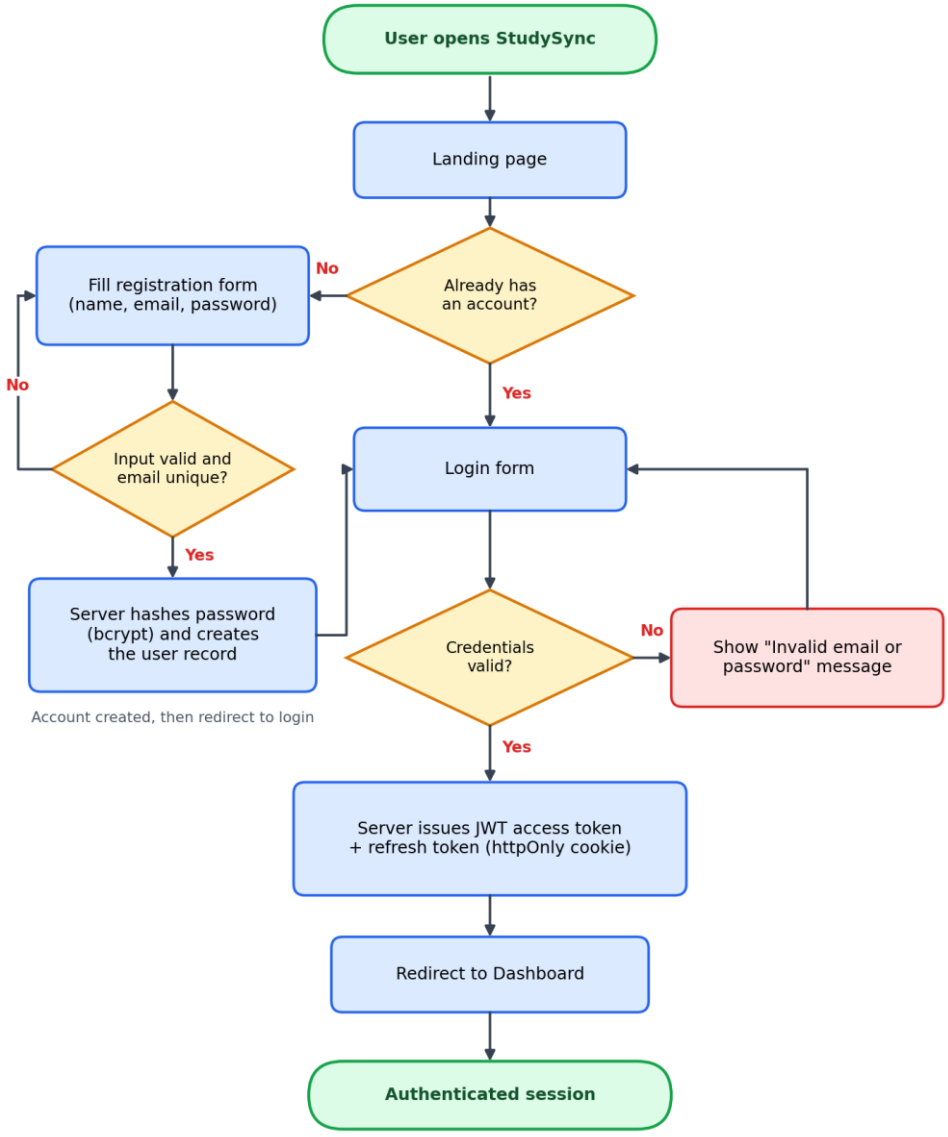



*Figure 1: Registration and login flow*

## 3.2 Group collaboration

From the dashboard a user can create a group, join one, or open one they already belong to. All three paths lead to the group workspace, where they can upload resources, discuss, or schedule sessions. Each action triggers the notification service, which stores a notification for every member and pushes a real-time event.

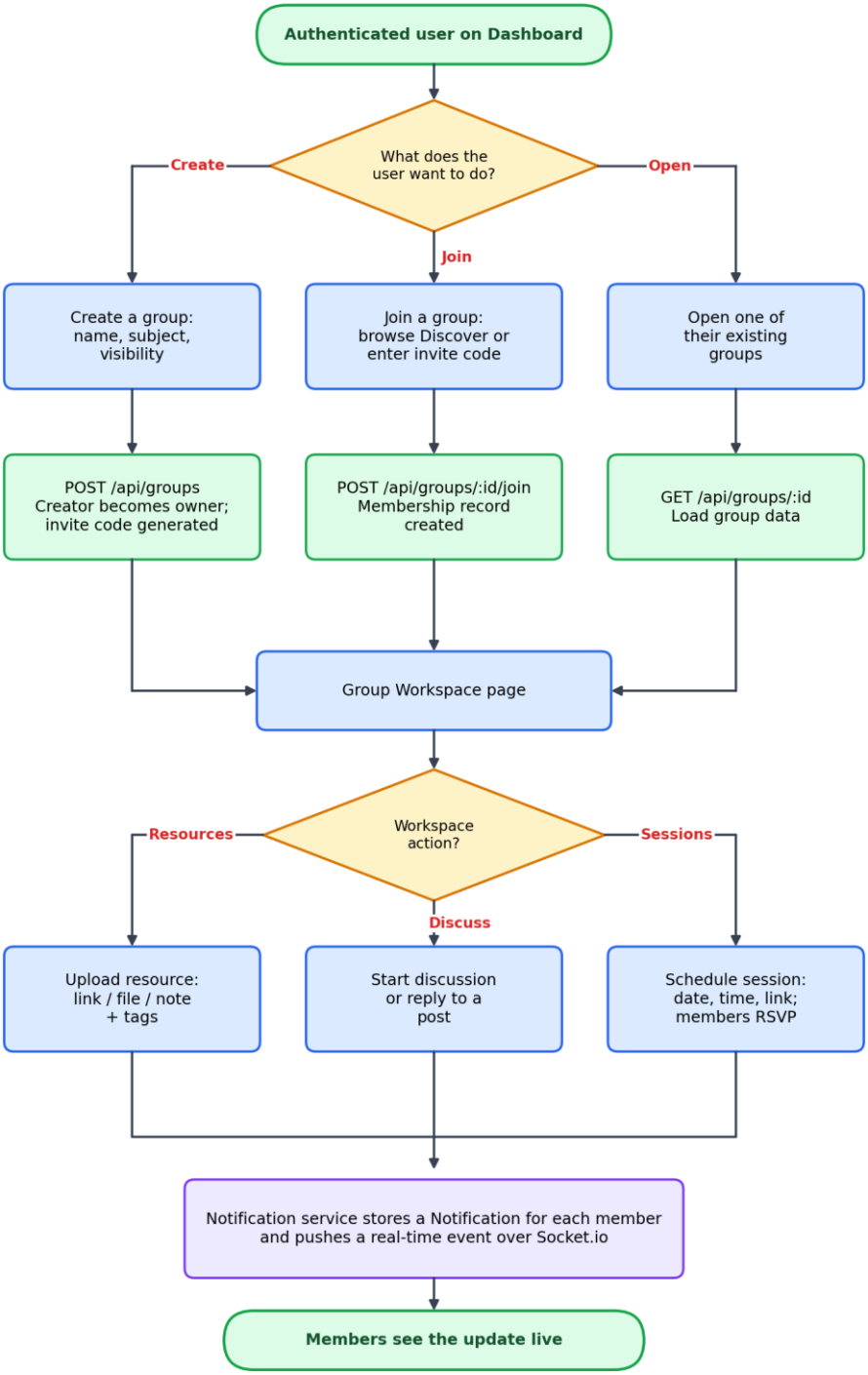



*Figure 2: Group creation, joining and collaboration flow*

# 4. Wireframes

The wireframes are low-fidelity, greyscale layouts that focus on structure and behaviour rather than visual style. Red numbered markers refer to the annotation tables below each screen. All screens share a top navigation bar and a left sidebar, which keeps navigation predictable and reduces the learning curve.

## 4.1 Login and registration

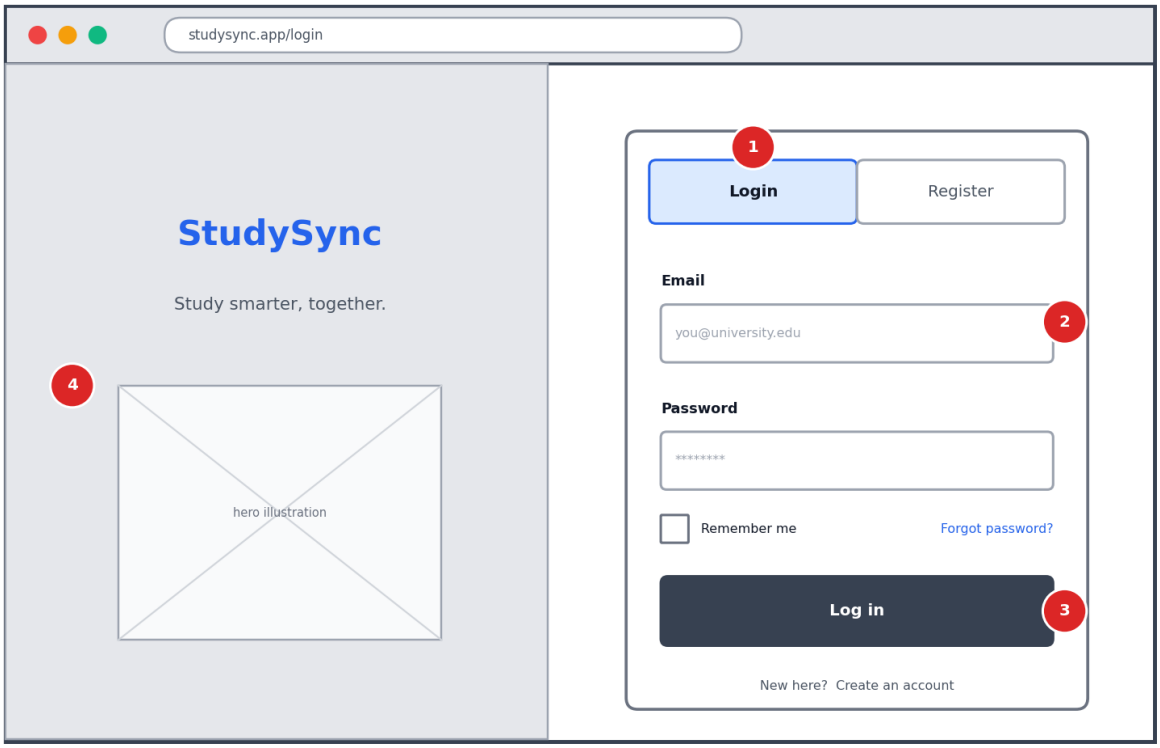



*Figure 3: Login and registration screen*

| No. | Element | Behaviour |
|---|---|---|
| 1 | Login / Register tabs | Switch between the two forms without leaving the page; Register adds name and confirm-password fields |
| 2 | Email and password fields | Inline validation on blur; errors shown under each field; password strength hint on Register |
| 3 | Primary button | Disabled while the request is in progress; shows a spinner; generic error on failure |
| 4 | Brand panel | Hidden on small screens so the form takes the full width |

## 4.2 Dashboard

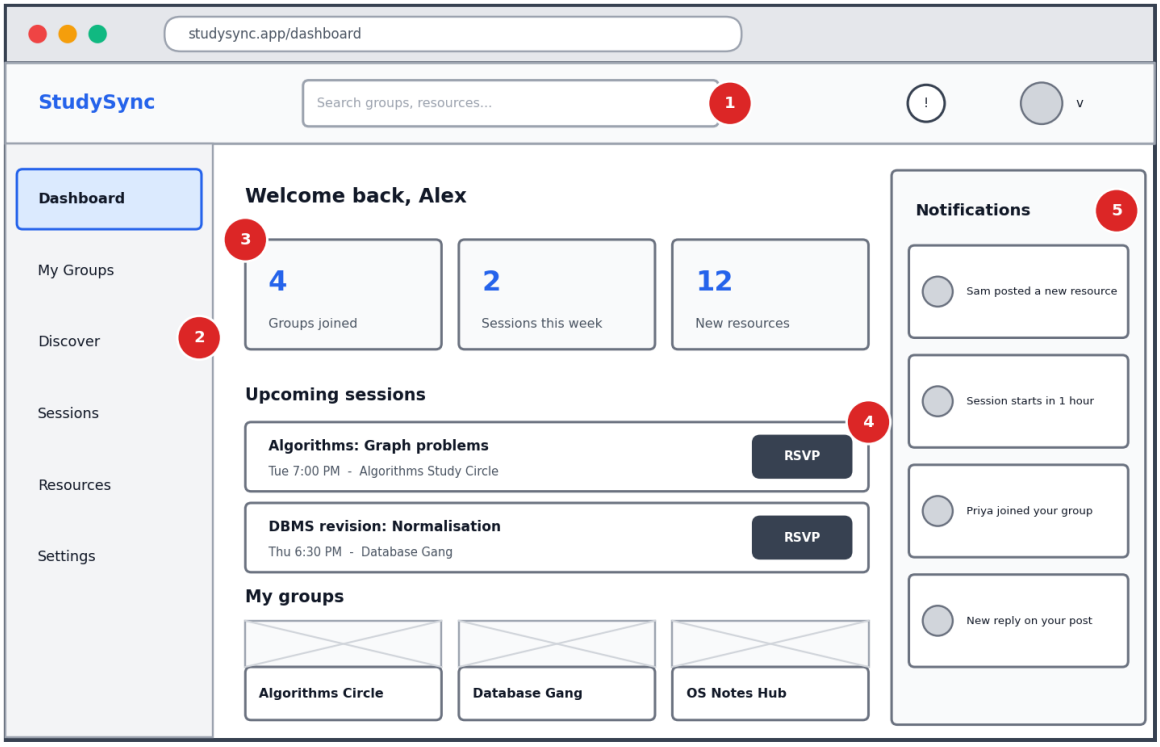



*Figure 4: Dashboard*

| No. | Element | Behaviour |
|---|---|---|
| 1 | Top bar and global search | Searches group names and resource titles; results appear in a dropdown |
| 2 | Sidebar navigation | Highlights the active page; collapses into a menu icon on mobile |
| 3 | Summary cards | Groups joined, sessions this week and new resources; each card links to its list |
| 4 | Upcoming sessions | Next sessions across all groups with one-click RSVP |
| 5 | Notifications panel | Latest items update live through Socket.io; unread items are emphasised and clicking opens the related page |

## 4.3 Group workspace

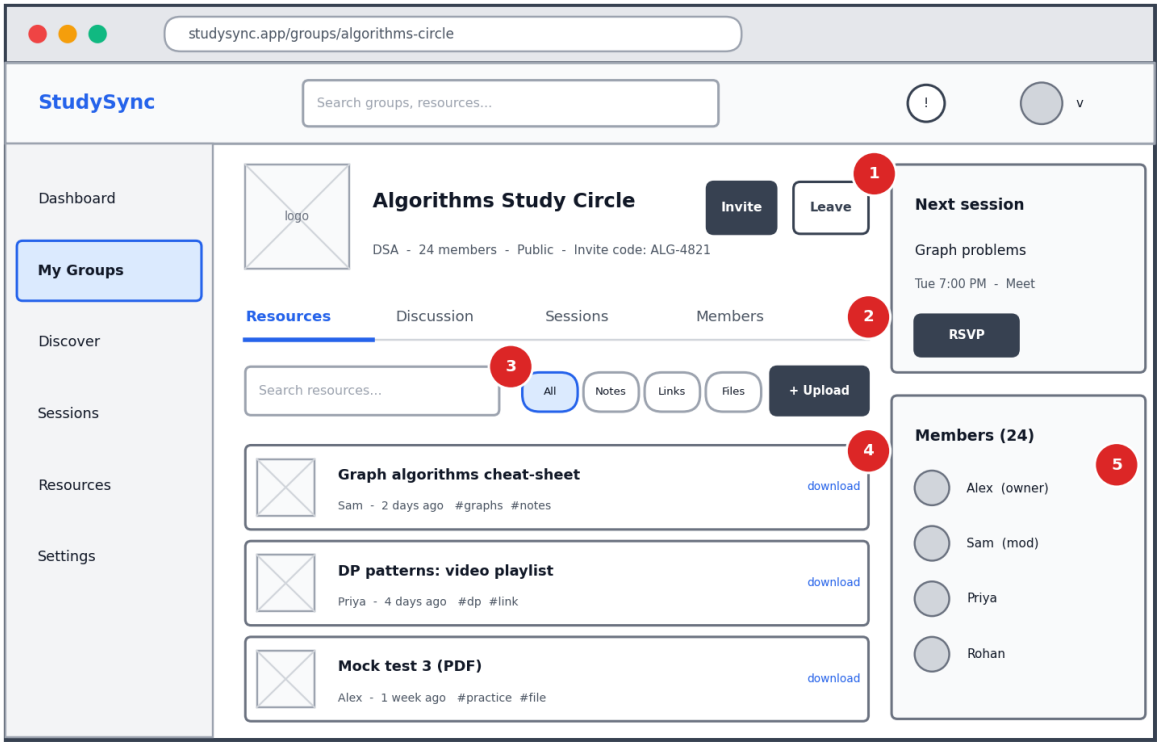



*Figure 5: Group workspace (Resources tab)*

| No. | Element | Behaviour |
|---|---|---|
| 1 | Group header | Name, subject, member count, visibility and invite code; Invite and Leave actions depend on role |
| 2 | Tabs | Resources, Discussion, Sessions and Members share the same page shell so switching is instant |
| 3 | Toolbar | Search, type filter chips (All, Notes, Links, Files) and the Upload button |
| 4 | Resource list | Shows title, uploader, age and tags; owners, moderators and the uploader can delete |
| 5 | Side panel | Next session with RSVP and the member list with roles |

## 4.4 Discover groups

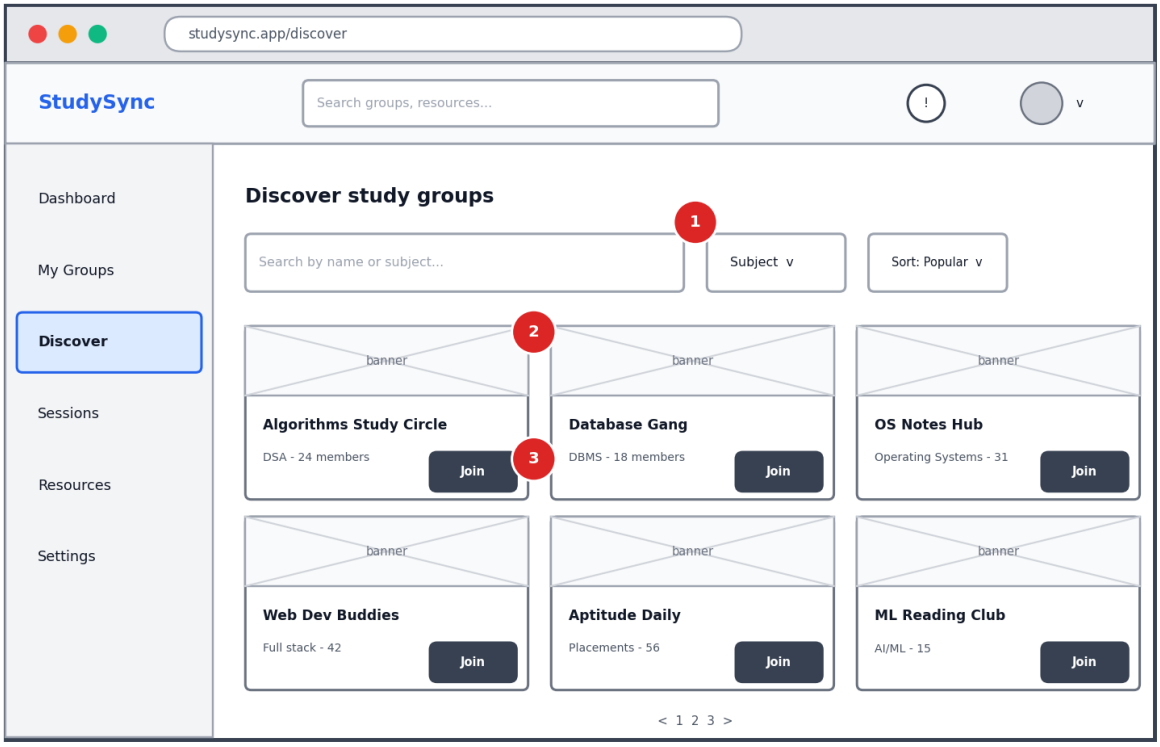



*Figure 6: Discover groups*

| No. | Element | Behaviour |
|---|---|---|
| 1 | Search and filters | Debounced search, subject dropdown and sort order (popular, newest); results are paginated |
| 2 | Group card | Banner, name, subject and member count; clicking opens a preview for public groups |
| 3 | Join button | Joins a public group immediately; private groups ask for an invite code instead |

## 4.5 Mobile layout

On phones the sidebar becomes a bottom navigation bar, summary cards shrink to a single row, and the Upload action becomes a floating button. Touch targets are at least 44 px high.

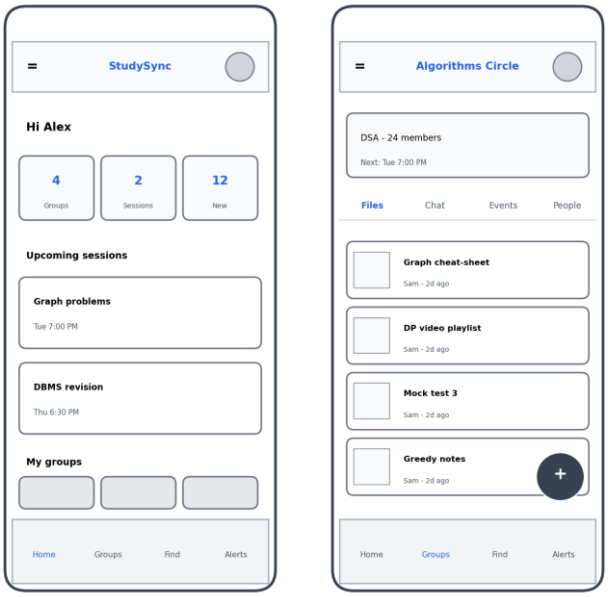



*Figure 7: Mobile views of the dashboard and group workspace*

## 4.6 Design principles

- **Consistency:** the same shell, spacing scale and button styles on every page.
- **Clarity:** one primary action per screen, shown as the dark filled button.
- **Feedback:** loading skeletons, toast messages and inline errors for every request.
- **Accessibility:** labels on all inputs, keyboard-reachable controls and sufficient contrast.

# 5. Technical Architecture

StudySync follows a three-tier architecture: a React single-page application (client tier), a Node.js and Express REST API (application tier), and MongoDB Atlas (data tier). A WebSocket channel runs beside the REST API for real-time notifications.

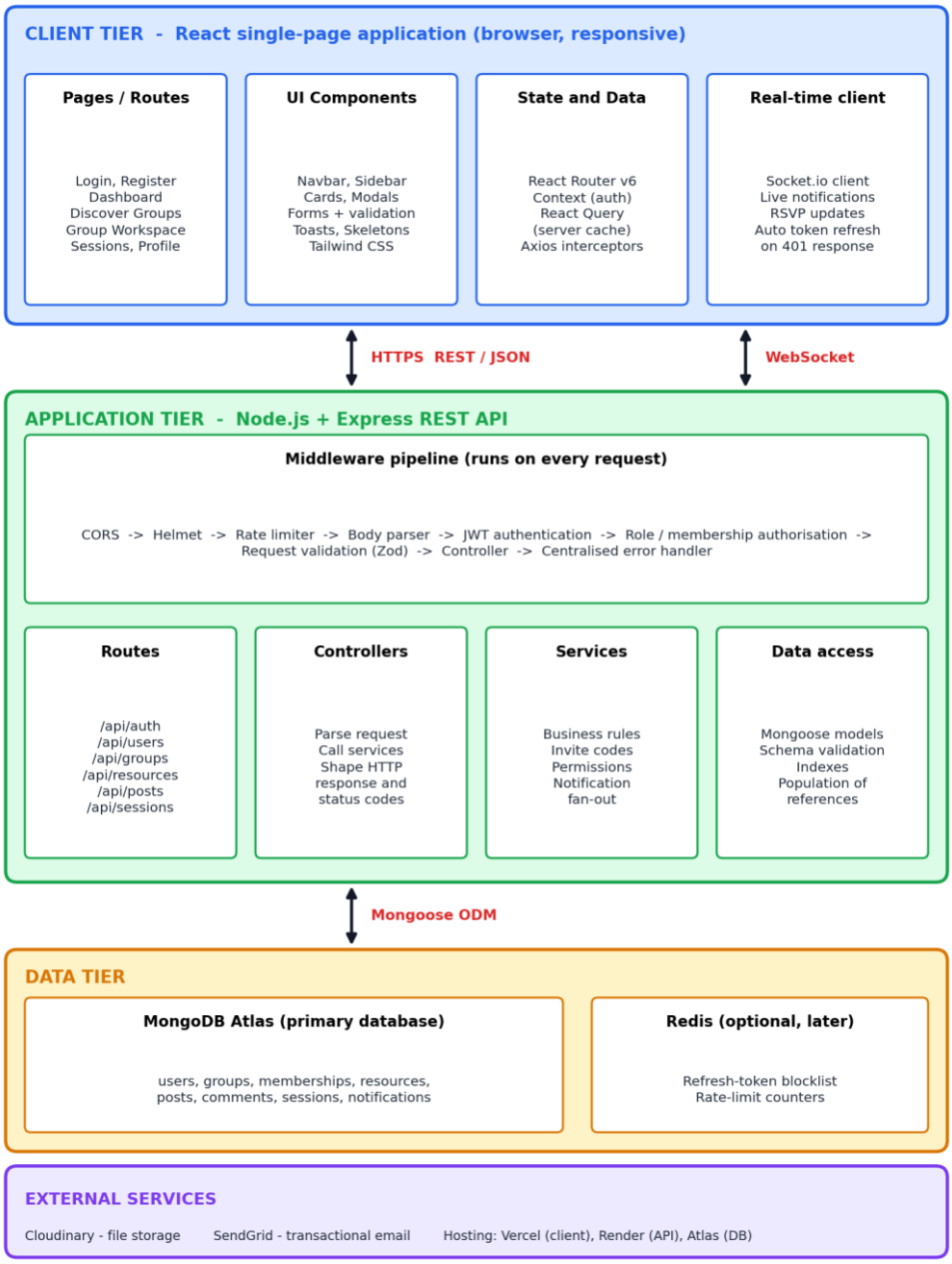



*Figure 8: System architecture*

## 5.1 Component responsibilities

| Component | Responsibility |
|---|---|
| Pages and routes | Top-level screens (Login, Dashboard, Discover, Group Workspace, Sessions, Profile). Protected routes redirect unauthenticated users to Login. |
| UI components | Reusable pieces such as navbar, sidebar, cards, modals, forms and toasts. |

| Component | Responsibility |
|---|---|
| State and data layer | React Context stores the signed-in user; React Query caches API responses; an Axios interceptor attaches the access token and refreshes it on a 401 response. |
| Middleware pipeline | CORS, Helmet security headers, rate limiter, body parser, JWT verification, role or membership checks, Zod validation and a final error handler. |
| Routes | Map URLs and HTTP verbs to controllers (for example /api/groups). |
| Controllers | Read the request, call a service and send the HTTP response. They contain no business rules. |
| Services | Business logic: generating invite codes, checking permissions, creating notifications and sending them to members. |
| Data access (models) | Mongoose schemas, validation, indexes and population of references. |
| Socket.io server | Authenticates the connection with the access token, places each user in a room per group and a room for themselves, and emits notification events. |
| External services | Cloudinary stores uploaded files; SendGrid sends optional emails. |

## 5.2 Request lifecycle example: uploading a resource

1. The user submits the upload form; React Hook Form validates title, type and URL or file.
2. Axios sends POST /api/groups/:id/resources with the access token in the Authorization header. Files go to Cloudinary first and the returned URL is sent to the API.
3. The middleware pipeline checks rate limits, verifies the JWT and loads the user.
4. A membership check confirms that the user belongs to the group.
5. Zod validates the body; invalid data returns 400 with field-level messages.
6. The controller calls the resource service, which saves the document through the Mongoose model.
7. The notification service creates a Notification for each other member and emits a Socket.io event to the group room.
8. The API returns 201 with the new resource; React Query updates the list and other members see the live notification.

## 5.3 Security design

- Passwords are hashed with bcrypt (cost factor 12); plain-text passwords are never stored or logged.
- Access tokens last 15 minutes; refresh tokens last 7 days, are stored in an httpOnly, secure, SameSite cookie and are rotated on use.
- Authorisation is checked on the server for every group route: private data is returned only to members, and owner-only actions check the membership role.
- Helmet sets secure headers; CORS allows only the deployed front-end origin; the rate limiter is stricter on /api/auth.
- Zod schemas validate all input, and express-mongo-sanitize removes operators such as $ from request bodies to prevent NoSQL injection.
- Uploads are restricted by type and size; secrets live in environment variables and are never committed to Git.

## 5.4 Project structure

```
studysync/
  client/                    React + Vite application
    src/pages/               route-level screens
    src/components/          reusable UI components
    src/hooks/               custom hooks (useAuth, useGroups ...)
    src/api/                 Axios instance and API functions
    src/context/             AuthContext, SocketContext
  server/                    Node.js + Express API
    src/routes/              URL to controller mapping
    src/controllers/         request and response handling
    src/services/            business logic
    src/models/              Mongoose schemas
    src/middleware/          auth, validate, error handler
    src/sockets/             Socket.io setup and events
    tests/                   Jest and Supertest tests
  docs/                      this report, diagrams, API collection
```

## 5.5 Deployment

The client is built by Vite and deployed to Vercel; the API runs as a Render web service; MongoDB Atlas hosts the database. Environment variables hold the database URI, JWT secrets and Cloudinary keys. A GitHub Actions workflow runs linting and tests on every pull request before merging into the main branch.

# 6. REST API Design

The API follows REST conventions: nouns in URLs, HTTP verbs for actions, JSON bodies, and standard status codes (200, 201, 400, 401, 403, 404, 409, 422, 429, 500). Group content is nested under the group (for example /api/groups/:id/resources) to make the ownership of data explicit. Every list endpoint accepts page and limit query parameters.

Success responses use the shape { success: true, data }. Errors use { success: false, message, errors? } so the front end can handle them in one place.

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| POST | `/api/auth/register` | Public | Create an account |
| POST | `/api/auth/login` | Public | Verify credentials and issue tokens |
| POST | `/api/auth/refresh` | Cookie | Issue a new access token and rotate the refresh token |
| POST | `/api/auth/logout` | User | Clear the refresh cookie |
| GET | `/api/users/me` | User | Get the current profile |
| PATCH | `/api/users/me` | User | Update name or avatar |
| GET | `/api/users/me/dashboard` | User | Counts, upcoming sessions and recent activity |
| GET | `/api/groups` | User | Discover public groups (search, subject, sort, page) |
| POST | `/api/groups` | User | Create a group; creator becomes owner |
| GET | `/api/groups/mine` | User | Groups the user belongs to |
| GET | `/api/groups/:id` | Member / public | Group details |
| PATCH | `/api/groups/:id` | Owner | Edit group details |
| DELETE | `/api/groups/:id` | Owner | Delete the group and its content |
| POST | `/api/groups/:id/join` | User | Join a public group |
| POST | `/api/groups/join-by-code` | User | Join any group with an invite code |
| DELETE | `/api/groups/:id/leave` | Member | Leave a group (owner must transfer ownership first) |
| GET | `/api/groups/:id/members` | Member | List members and roles |
| PATCH | `/api/groups/:id/members/:userId` | Owner | Change a member's role |
| DELETE | `/api/groups/:id/members/:userId` | Owner / mod | Remove a member |
| GET | `/api/groups/:id/resources` | Member | List resources (filter by type, tag, search) |
| POST | `/api/groups/:id/resources` | Member | Add a resource |
| DELETE | `/api/resources/:id` | Uploader / mod / owner | Delete a resource |

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| GET | `/api/groups/:id/posts` | Member | List discussion posts |
| POST | `/api/groups/:id/posts` | Member | Create a post |
| GET | `/api/posts/:id` | Member | Post with its comments |
| POST | `/api/posts/:id/comments` | Member | Add a comment |
| GET | `/api/groups/:id/sessions` | Member | List study sessions |
| POST | `/api/groups/:id/sessions` | Mod / owner | Schedule a session |
| PUT | `/api/sessions/:id/rsvp` | Member | Set RSVP to going, maybe or no |
| GET | `/api/notifications` | User | List notifications (unread first) |
| PATCH | `/api/notifications/:id/read` | User | Mark one as read |
| PATCH | `/api/notifications/read-all` | User | Mark all as read |

## 6.1 Example: create a resource

**Request** POST /api/groups/64f1.../resources

```
{
  "title": "Graph algorithms cheat-sheet",
  "type": "link",
  "url": "https://example.com/graphs.pdf",
  "tags": ["graphs", "notes"]
}
```

**Response** 201 Created, with { success: true, data: { _id, title, type, url, tags, uploadedBy, createdAt } }. Possible errors: 401 (no or expired token), 403 (not a member), 422 (validation failed).

# 7. Database Design

StudySync uses MongoDB with eight collections. Relationships are stored as ObjectId references and resolved with Mongoose populate, except for small arrays that are always read with their parent (resource tags and session attendees), which are embedded.

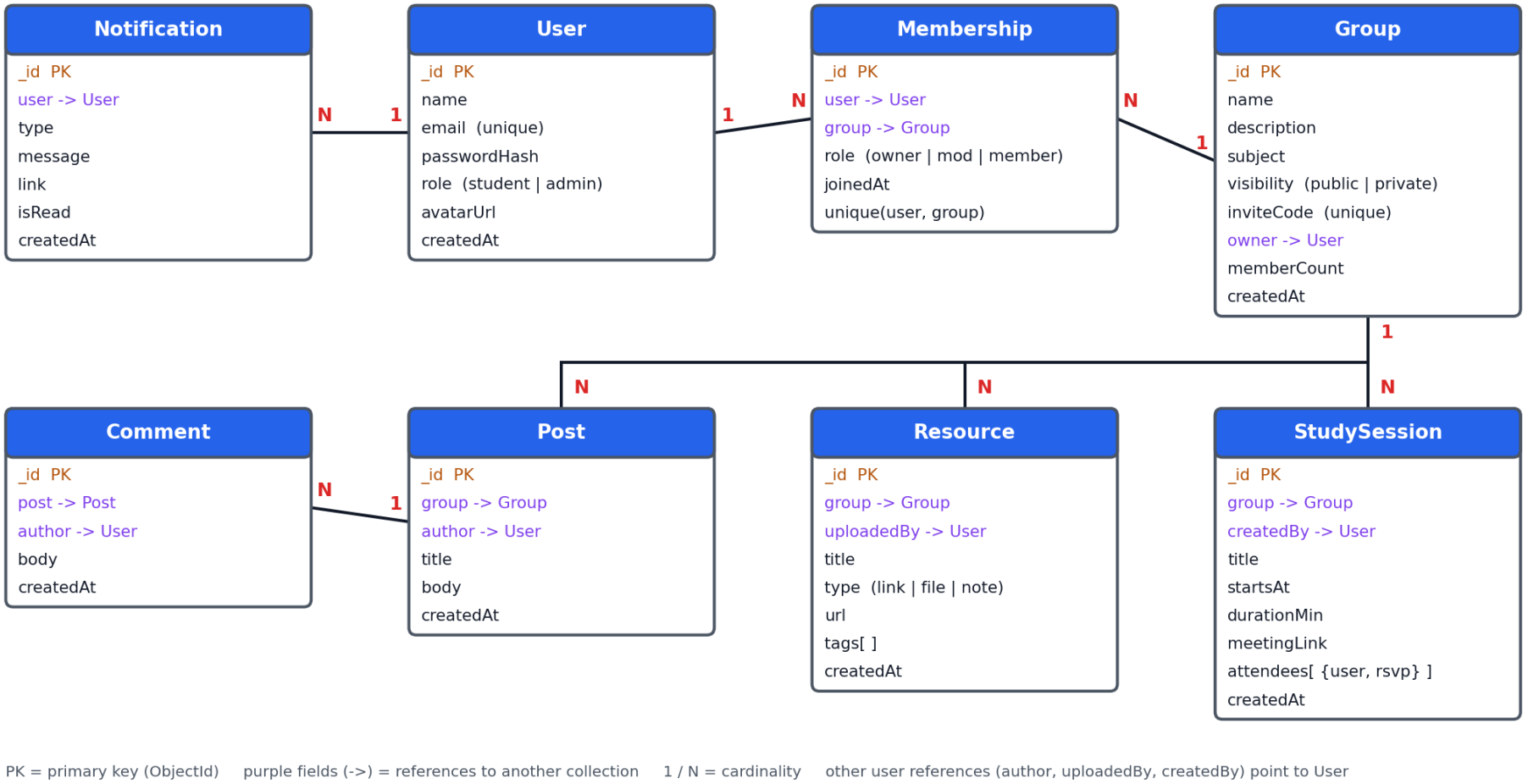



*Figure 9: Entity relationship diagram (collections and references)*

## 7.1 Collections and indexes

| Collection | Purpose | Relationships | Indexes |
|---|---|---|---|
| users | Accounts and profile data | Referenced by every other collection | email (unique) |
| groups | Study groups and their settings | owner references users; one group has many memberships, resources, posts and sessions | inviteCode (unique); text index on name and subject; visibility + memberCount for discovery |
| memberships | Join table between users and groups, holding the role | Many-to-many between users and groups | compound unique (user, group); group for member lists |
| resources | Shared links, files and notes | Belongs to one group; uploadedBy references users | group + createdAt; tags |
| posts | Discussion threads | Belongs to one group; author references users; has many comments | group + createdAt |
| comments | Replies to posts | Belongs to one post; author references users | post + createdAt |
| studysessions | Scheduled study sessions with RSVPs | Belongs to one group; createdBy references users; attendees embedded | group + startsAt; attendees.user |
| notifications | Per-user notification feed | Belongs to one user | user + isRead + createdAt |

## 7.2 Relationship summary

- **User to Group (many-to-many)** through Membership. A separate collection is used because the relationship carries data (role, join date) and must be queried from both directions.
- **Group to Resource, Post and StudySession (one-to-many).** Each child stores a group reference, which keeps documents small and allows paginated queries.
- **Post to Comment (one-to-many).** Comments are separate documents so that long threads do not hit document-size limits and can be paginated.
- **User to Notification (one-to-many).** Notifications are written once per recipient so that read state is tracked individually.
- **StudySession attendees** are embedded as { user, rsvp } because they are always read together with the session and the list is small.

## 7.3 Sample Mongoose schema

```
const membershipSchema = new Schema({
  user:  { type: ObjectId, ref: 'User',  required: true },
  group: { type: ObjectId, ref: 'Group', required: true },
  role:  { type: String, enum: ['owner','moderator','member'], default: 'member' },
  joinedAt: { type: Date, default: Date.now },
});
membershipSchema.index({ user: 1, group: 1 }, { unique: true });
```

## 7.4 Data integrity rules

- Deleting a group removes its memberships, resources, posts, comments and sessions in one service function.
- A group must always have one owner; the owner must transfer ownership before leaving.
- Duplicate memberships are impossible because of the compound unique index.
- Private groups are never returned by the discovery endpoint.

# 8. Design Decisions and Rationale

| Decision | Alternatives | Rationale and trade-off |
|---|---|---|
| Single-page React app with a separate REST API | Server-rendered pages (EJS, Next.js) | A clear API boundary allows independent development and testing and could later serve a mobile app. The trade-off is weaker default SEO, which matters little for a login-based product. |
| MongoDB | PostgreSQL | Resources and posts vary in shape and are read as whole documents, and Mongoose speeds up the MVP. Relational integrity is handled with unique indexes and service-level checks; a move to SQL stays possible because data access is isolated in models. |
| Membership as its own collection | Array of member IDs inside Group | Supports roles, join dates and queries such as 'all groups for a user'. Unbounded arrays inside a group document would grow without limit. |
| Layered backend (routes, controllers, services, models) | Fat route handlers | Keeps business rules testable and reusable (for example, notification fan-out is used by several controllers). |
| JWT access token plus rotating refresh cookie | Server sessions; tokens in localStorage | Short-lived tokens limit the impact of theft; httpOnly cookies are not readable by scripts, reducing XSS risk. |
| Socket.io rooms per group | Polling every few seconds | Instant updates with less network traffic. A fallback to REST fetching keeps the app usable if the socket drops. |
| Files stored on Cloudinary, only URLs in the database | Store files in MongoDB (GridFS) or on server disk | Cloud hosts have ephemeral disks, and a CDN serves files faster. The trade-off is a dependency on a third-party service. |
| Pagination on every list | Return all results | Protects performance as groups grow and keeps responses small. |
| Mobile-first Tailwind layout | Desktop-first with media queries | Most students use phones, so small screens get the primary design. |
| MoSCoW prioritisation for scope | Build everything in the idea list | Guarantees that Must items form a working MVP before optional features such as email reminders. |

# 9. Risks, Roadmap and Conclusion

## 9.1 Risks and mitigations

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Scope creep beyond the MVP | High | High | Stick to MoSCoW priorities; log new ideas in a backlog instead of building them immediately |
| Unauthorised access to private groups | Medium | High | Central membership middleware, API tests for every protected route |
| Real-time features add complexity | Medium | Medium | Build REST first; add Socket.io after core flows work; fall back to refetching |
| File storage cost or abuse | Medium | Medium | Size and type limits, per-user upload limits, links preferred over files |
| Free hosting tiers sleep or have limits | High | Low | Show loading state on first request; optional keep-alive ping; upgrade if needed |
| Time overrun | Medium | Medium | Work in small vertical slices and merge to main frequently |

## 9.2 Proposed development roadmap

| Phase | Focus | Deliverables |
|---|---|---|
| 1 | Setup and authentication | Repositories, project structure, database connection, register/login/refresh/logout, protected routes |
| 2 | Groups and membership | Create, discover, join, leave, roles and invite codes; Discover and Group pages |
| 3 | Resources and discussion | Resource CRUD with tags and filters, Cloudinary upload, posts and comments |
| 4 | Sessions and notifications | Scheduling, RSVP, notification service, Socket.io, dashboard |
| 5 | Quality and release | Tests, accessibility and performance fixes, deployment, final documentation |

## 9.3 Conclusion

This document defines StudySync from concept to architecture: the problem it solves, the users it serves, prioritised requirements, user flows, wireframes for the main screens, a layered three-tier architecture, a REST API contract and a MongoDB schema with clear relationships. Each major choice is recorded with alternatives and trade-offs. With this plan in place, implementation can proceed in small, testable slices, beginning with the authentication module.